

- RAG Learning

1.1 Systems Architecture

OCR GCSE Computer Science (J277) — Paper 1

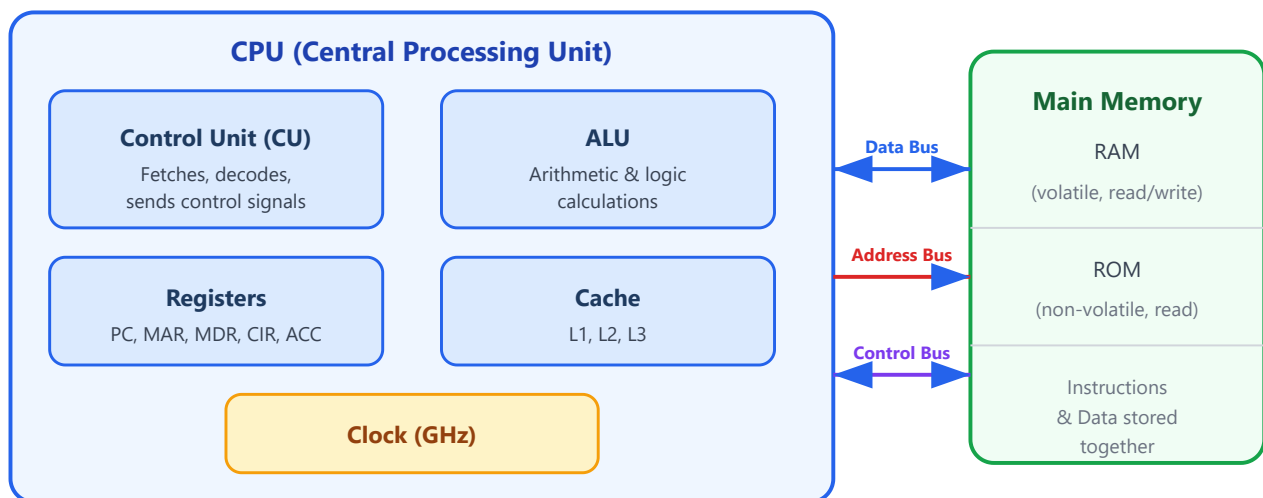
1.1.1 Von Neumann Architecture and CPU Components

The **Von Neumann architecture** is a computer design where data and instructions are stored in the **same memory** and are processed **sequentially** (one instruction at a time). This is the basis of virtually all modern general-purpose computers. The key insight is that programs are data — they can be loaded, modified, and replaced without physically rewiring the machine.

Before Von Neumann's design (proposed in 1945), early computers like ENIAC had to be physically rewired to perform different tasks. Von Neumann realised that if the program itself were stored in memory alongside the data, the computer could be reprogrammed simply by loading different instructions — no rewiring needed. This concept is called the **stored program concept** and remains the foundation of computing today.

The alternative is the **Harvard architecture**, where instructions and data have **separate memory stores** and **separate buses**. This allows instructions and data to be fetched simultaneously, which is why it is used in some embedded systems and digital signal processors where speed is critical. However, the Von Neumann design is simpler and more flexible, which is why general-purpose computers use it.

Von Neumann Architecture



Data and instructions share the same memory (stored program concept)
Connected by a shared system bus (address + data + control)

Key CPU Components

The **CPU (Central Processing Unit)** is the brain of the computer. It fetches instructions from memory, decodes them to understand what action is required, and then executes them. The CPU is made up of several key components that work together:

Component	Function	Detail
ALU (Arithmetic Logic Unit)	Performs all arithmetic calculations and logical comparisons	Arithmetic: add, subtract, multiply, divide. Logic: AND, OR, NOT, comparisons (equal, greater than, less than). Results are stored in the accumulator. The ALU is the only part of the CPU that actually performs calculations on data.
Control Unit (CU)	Coordinates all CPU operations	Fetches instructions from memory, decodes them into operations, and directs other components (ALU, registers, buses) to carry them out. It sends control signals via the control bus. The CU does not process data itself — it orchestrates the work of other components, like a conductor directing an orchestra.
Registers	Ultra-fast storage inside the CPU	Small number of storage locations that hold data, addresses, or instructions currently being used. Much faster than RAM (accessed in a single clock cycle). Each register typically holds

		one word of data (e.g., 32 or 64 bits). There are only a handful of registers, each with a specific purpose.
Cache	Fast memory between CPU and RAM	Stores copies of frequently accessed data and instructions from RAM so the CPU does not have to wait for slower RAM. L1 cache is fastest and smallest (per core, ~32-64 KB); L2 is larger but slightly slower (~256 KB-1 MB); L3 is largest and shared between cores (~4-32 MB). Cache is made of SRAM (faster than the DRAM used in main memory).
Clock	Synchronises all CPU operations	An electronic oscillator that generates a regular pulse. Each pulse is one clock cycle , and one or more stages of an instruction happen per cycle. Clock speed is measured in GHz (billions of cycles per second). A 3.5 GHz processor performs 3.5 billion clock cycles every second. The clock ensures all components work in step with each other.

Buses — The CPU's Communication Highways

The CPU communicates with memory and I/O devices via a **system bus** consisting of three sub-buses. Think of buses as motorways connecting different parts of the computer:

Bus	Carries	Direction	Analogy
Address Bus	Memory addresses — tells memory which location to read from or write to	Unidirectional (CPU → memory only)	Like a postcode on a letter — it tells the postal system WHERE to deliver, but the letter itself travels separately
Data Bus	Actual data being transferred between CPU, memory, and I/O devices	Bidirectional (data flows both ways)	Like the letter inside the envelope — the actual content being sent or received
Control Bus	Control signals: read, write, clock, interrupt, bus request/grant	Bidirectional	Like traffic lights controlling the flow — it tells components when to send, receive, read, or write

The **width of the address bus** determines how many unique memory locations the CPU can address. A 32-bit address bus can address $2^{32} = 4,294,967,296$ locations (4 GB). A 64-bit address bus can theoretically address 2^{64} locations (16 exabytes). The **width of the data bus** determines how much data can be transferred in a single operation — a wider data bus means more data per transfer, improving performance.

Exam Tip: The address bus is **unidirectional** (CPU sends addresses to memory, never the other way). The data bus is **bidirectional** (data can be read FROM memory or written TO memory). This is a commonly examined distinction. Remember: the address bus width determines how much memory can be addressed; the data bus width determines how much data can be transferred at once.

1.1.2 Registers

Registers are the **fastest storage locations** in a computer, located directly inside the CPU. They are used to hold data, addresses, and instructions that the CPU is currently working with. Each register has a specific role in the fetch-decode-execute cycle.

Register	Full Name	Holds	Purpose During the FDE Cycle
PC	Program Counter	Address	Holds the address of the NEXT instruction to be fetched. Incremented by 1 after each fetch (unless a branch instruction changes it to a different address). The PC is what allows the CPU to know where it is in a program.
MAR	Memory Address Register	Address	Holds the memory address the CPU wants to read from or write to. Connected to the address bus. The MAR is like a request slip sent to memory: "give me the data at this location". The MAR is loaded from the PC during fetch, or from the CIR operand during execute.
MDR	Memory Data Register	Data	Holds the data just read from memory, or the data about to be written to memory. Connected to the data bus. Also called the Memory Buffer Register (MBR). It

			acts as a buffer between the CPU and memory — all data passing between them goes through the MDR.
ACC	Accumulator	Data	Stores the result of the most recent ALU operation (arithmetic or logic). Acts as the primary working register for calculations. For example, after an ADD instruction, the sum is stored here.
CIR	Current Instruction Register	Data (instruction)	Holds the instruction currently being decoded and executed . The CU splits its contents into the opcode (what to do, e.g., ADD, LOAD, STORE, BRANCH) and the operand (the data or address to use). The CIR is needed so the MDR is free to be used for data access during the execute phase.

Exam Tip: Always distinguish between registers that hold **addresses** (PC, MAR) and those that hold **data** (MDR, ACC, CIR). The PC holds the address of the *next* instruction, not the current one. The MAR holds addresses; the MDR holds data. A common mistake is confusing the MAR and MDR — remember: MAR = Address, MDR = Data. This distinction is required for full marks.

How Registers Connect to the Buses

Understanding which registers connect to which buses is important:

- The **MAR** connects to the **address bus** — it sends the address of the memory location to be accessed
- The **MDR** connects to the **data bus** — it receives data read from memory or sends data to be written
- The **CU** uses the **control bus** to send signals such as "memory read" or "memory write"
- The **PC**, **ACC**, and **CIR** are internal registers — they do not connect directly to external buses but communicate with other registers inside the CPU

Why Each Register Exists — The Architectural Reason

Understanding the *purpose* behind each register, rather than just its function, gives you a much stronger answer in 4- and 6-mark questions:

- **PC exists** because the CPU must always know the address of the *next* instruction to fetch. Without the PC, the CPU would have no way to keep track of where it is in a program. The PC is modified by branch instructions, which is the hardware mechanism behind IF statements, loops, and function calls.
- **MAR exists** to hold the address being sent on the address bus long enough for memory to respond. Memory access takes time (RAM takes ~50-100 ns); the MAR decouples the CPU from memory timing — the CPU can load the MAR and then do other internal work while waiting. Without the MAR, the address bus would need to be driven continuously, tying up the CPU.
- **MDR exists** as a buffer between the fast CPU and the relatively slow RAM. Data arriving from memory over the data bus is placed in the MDR first, then transferred internally within the CPU at CPU speed. This decoupling means the CPU does not have to wait at the exact moment data arrives on the bus. Without the MDR, every memory access would require the CPU to halt and hold the bus until the transfer was complete.
- **CIR exists** so that the instruction being decoded and executed is preserved while the MDR is reused. During the EXECUTE phase of a LOAD instruction, the MAR and MDR are needed again to access the data address. If the instruction were still in the MDR, it would be overwritten. Moving the instruction to the CIR during FETCH frees the MDR for data transfers during EXECUTE.
- **ACC exists** as the primary working register for the ALU. The ALU is wired to take input from the ACC and return results to the ACC. Without a dedicated accumulator, every intermediate calculation result would need to be written back to memory, which would be extremely slow. The ACC keeps intermediate results on-chip, inside the CPU, at register speed.

Exam Tip: The MAR and MDR exist to **decouple** the CPU from memory timing. Memory is much slower than the CPU; these registers act as buffers so the CPU does not have to pause while memory responds. This is why the instruction is moved from MDR to CIR during fetch — to free the MDR for use during execute. This level of reasoning (not just "what" but "why") is what distinguishes a full-mark answer from a partial one.

1.1.3 The Fetch-Decode-Execute (FDE) Cycle

The CPU continuously repeats three stages for every instruction. This cycle runs billions of times per second and is the fundamental operation of every Von Neumann computer. Everything a computer does — from displaying text to playing games — is ultimately the result of billions of FDE cycles.

1. Fetch

The purpose of the fetch phase is to retrieve the next instruction from memory and bring it into the CPU.

1. The address in the **PC** is copied to the **MAR**. (The CPU now knows which memory address to read from.)
2. The **PC is incremented** by 1 ($PC = PC + 1$), so it now points to the next instruction. (This happens immediately so the PC is ready for the next cycle.)
3. The address in the MAR is sent along the **address bus** to memory.
4. The CU sends a **memory read** signal on the control bus. (This tells memory to send back the data at the requested address.)
5. The instruction at that memory address is transferred along the **data bus** into the **MDR**.
6. The contents of the MDR are copied to the **CIR**. (This frees the MDR for use during the execute phase.)

Summary: PC → MAR → Memory → MDR → CIR. PC incremented.

2. Decode

The purpose of the decode phase is to work out what the instruction means and prepare the CPU to execute it.

1. The **Control Unit** examines the instruction in the CIR.
2. It splits the instruction into the **opcode** (the operation to perform, e.g., ADD, LOAD, STORE, BRANCH) and the **operand** (the data or address to use).
3. The CU determines which control signals to send and to which components. For example, if the opcode is ADD, the CU knows it needs to activate the ALU and route data appropriately.

3. Execute

The purpose of the execute phase is to carry out the decoded instruction.

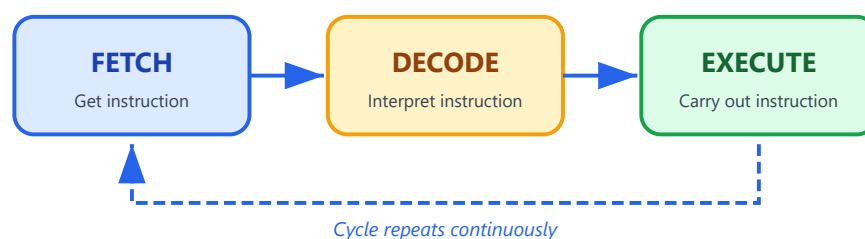
1. The decoded instruction is carried out. This could be:
 - An **arithmetic/logic operation** — the ALU performs a calculation (e.g., ADD, SUBTRACT, COMPARE) and the result goes to the ACC

- A **data transfer** — load data from memory into a register (LOAD), or store register data to memory (STORE). For LOAD: the operand address goes to the MAR, memory is read, data comes back via data bus to MDR, then MDR contents go to ACC.
- A **branch** — load a new address into the PC, changing the sequence of execution. This is how loops and IF statements work at the hardware level.

2. The result may be stored in the ACC or written back to memory via the MDR.

3. The cycle then repeats from the fetch phase — the PC already points to the next instruction.

The Fetch-Decode-Execute Cycle



Worked Example — Full FDE Trace

This example shows exactly what happens in the CPU as three instructions are executed. Follow the state of each register at every step:

Memory:

Address 0:	LOAD 5	(load value at address 5 into ACC)
Address 1:	ADD 6	(add value at address 6 to ACC)
Address 2:	STORE 7	(store ACC into address 7)
Address 5:	10	(data)
Address 6:	25	(data)
Address 7:	0	(will receive result)

Cycle 1 (LOAD 5):

FETCH:

PC=0 copied to MAR	MAR=0
PC incremented	PC=1
Address 0 sent on address bus, CU sends "read" signal	
Memory sends contents of address 0 via data bus	MDR="LOAD 5"
MDR copied to CIR	CIR="LOAD 5"

DECODE:

CU splits CIR: Opcode=LOAD, Operand=5

CU knows: "I need to read from address 5 and put the value in ACC"

EXECUTE:

Operand 5 placed in MAR MAR=5

Memory[5] sent via data bus MDR=10

MDR copied to ACC ACC=10

Cycle 2 (ADD 6):

FETCH:

PC=1 copied to MAR MAR=1

PC incremented PC=2

Memory[1] via data bus MDR="ADD 6"

MDR copied to CIR CIR="ADD 6"

DECODE:

Opcode=ADD, Operand=6

CU knows: "I need to read from address 6 and add it to ACC"

EXECUTE:

Operand 6 placed in MAR MAR=6

Memory[6] via data bus MDR=25

ALU performs: $ACC + MDR = 10 + 25 = 35$

Result stored in ACC ACC=35

Cycle 3 (STORE 7):

FETCH:

PC=2 copied to MAR MAR=2

PC incremented PC=3

Memory[2] via data bus MDR="STORE 7"

MDR copied to CIR CIR="STORE 7"

DECODE:

Opcode=STORE, Operand=7

CU knows: "I need to write the ACC value to address 7"

EXECUTE:

Operand 7 placed in MAR MAR=7

ACC value copied to MDR MDR=35

CU sends "write" signal on control bus

MDR contents written to Memory[7]

Memory[7] now holds 35

Register Trace Table

A compact way to show the same information. This format is commonly used in exam questions:

Cycle	Phase	PC	MAR	MDR	CIR	ACC
-------	-------	----	-----	-----	-----	-----

1	Fetch	0 → 1	0	LOAD 5	LOAD 5	—
	Execute	1	5	10	LOAD 5	10
2	Fetch	1 → 2	1	ADD 6	ADD 6	10
	Execute	2	6	25	ADD 6	35
3	Fetch	2 → 3	2	STORE 7	STORE 7	35
	Execute	3	7	35	STORE 7	35

Exam Tip: When tracing the FDE cycle, always show the PC being **incremented during the fetch phase**, not after execute. A common mistake is forgetting to show the PC increment. Also note that during a LOAD execute, the MAR and MDR are used *again* (to access the data address), which is why the instruction was moved from MDR to CIR during fetch — if it stayed in MDR, it would be overwritten. This is a subtle but important point that examiners reward.

1.1.4 CPU Performance Factors

Three main factors affect how fast a CPU can process instructions. In exam questions, you must be able to explain **how** each factor improves performance AND its **limitations**.

Factor	How It Improves Performance	Limitations
Clock Speed	Measured in GHz. Higher clock speed = more FDE cycles per second. A 4 GHz processor does 4 billion cycles/second. Doubling clock speed roughly doubles throughput for single-threaded tasks. Going from 2 GHz to 4 GHz means twice as many instructions can be processed per second.	Higher clock speed generates more heat and uses more power . Physical limits exist (signal propagation, heat dissipation). Diminishing returns — modern CPUs hit thermal walls above ~5 GHz. Requires better cooling solutions (larger heatsinks, liquid cooling), which increases cost and size.

Number of Cores	Each core is an independent processor with its own ALU, CU, and registers. A quad-core CPU can run 4 threads simultaneously. Good for multi-tasking (running multiple programs) and multi-threaded applications (video editing, gaming, web servers).	Software must be written to use multiple threads to benefit. A single-threaded program uses only one core regardless of how many cores exist. Not all tasks can be divided into parallel sub-tasks. Overhead from coordinating between cores can reduce the expected speedup. Doubling cores does NOT double speed.
Cache Size	Cache stores frequently accessed data closer to the CPU. Larger cache = more data available without slow RAM access = fewer delays. L1 is fastest/smallest (~1 nanosecond access); L3 is slowest/largest (~10-20 ns). RAM access takes ~50-100 ns. A cache hit avoids the ~50-100 ns penalty of going to RAM.	Cache is very expensive per byte (SRAM vs DRAM). Diminishing returns as cache grows larger (larger caches are physically slower to search). Takes up physical space on the CPU die, leaving less room for other components. The relationship between cache size and performance is logarithmic, not linear.

Performance Factors — Summary for Comparisons

Question: "A user wants to improve their computer's performance for video editing. Explain how each CPU factor would help."

Clock speed: Higher clock speed means more FDE cycles per second, so each individual operation completes faster. Video encoding involves billions of calculations, each processed faster.

Cores: Video editing software is designed to be multi-threaded. A quad-core processor can encode 4 sections of video simultaneously, roughly quadrupling the speed compared to a single core. This is where cores provide the most benefit.

Cache: Larger cache stores more of the video frame data close to the CPU, reducing the number of slow RAM accesses needed. This reduces delays between calculations.

Best answer: For video editing, MORE CORES would provide the greatest benefit because video editing software is highly multi-threaded and can effectively utilise all available cores.

Exam Tip: When explaining CPU performance, always state *how* the factor helps AND its *limitations*. For example: "More cores allow multiple threads to execute simultaneously, improving performance for multi-threaded applications such as video editing. However, if the software is single-threaded, additional cores provide no benefit." This balanced answer scores higher than just stating the positive effect. Also know that increasing one factor cannot compensate for all workloads — a high clock speed with one core is great for single-threaded tasks but poor for multi-tasking.

Exam Tip: Cache holds copies of frequently accessed data and instructions from RAM. When the CPU requests data, it first checks the cache (**cache hit** = returned very quickly, ~1 ns). If not found (**cache miss**), the CPU must access slower RAM (~50-100 ns) and a copy is stored in cache for future use. Larger cache means more data stored close to the CPU, reducing the frequency of slow RAM accesses.

Exam Tip: Common exam question: "Suggest two ways to increase CPU performance." Never just say "faster processor" — name specific factors and explain *how* they help. Strong answers: (1) "Increase the clock speed — more FDE cycles are completed per second, so each instruction is processed faster." (2) "Increase the cache size — more data is stored close to the CPU, reducing the number of slow main memory accesses." Saying "use a faster computer" or "upgrade the CPU" will score zero marks.

Worked Exam Question — CPU Performance

Question: "A computer has a 3.2 GHz dual-core processor with 4 MB L3 cache. Explain two ways its performance could be improved." (4 marks)

Answer:

(1) Increase the clock speed (e.g., to 4.0 GHz).

The clock speed determines how many FDE cycles are completed per second. At 4.0 GHz, each core completes 4 billion cycles per second compared to 3.2 billion — an increase of 25%. Each individual instruction is processed faster, so the CPU completes more work in a given time.

Limitation: higher clock speed generates more heat and uses more power, requiring better (and more expensive) cooling.

(2) Increase the cache size (e.g., to 16 MB L3 cache).

Larger cache can store more frequently accessed data and instructions from RAM. When the CPU needs data, it is more likely to find it in fast cache (cache hit) rather than needing to access much slower RAM (cache miss). Reducing RAM accesses reduces delays between instructions.

Limitation: cache is expensive per byte, and very large caches have diminishing returns as they become physically slower to search.

Alternative valid answers: add more cores (enables parallel execution of multiple threads); upgrade to faster RAM (reduces the penalty of a cache miss).

1.1.5 Embedded Systems

An **embedded system** is a computer built into a larger device to perform one specific dedicated function. The program is stored in ROM (or flash memory) and runs automatically when the device is powered on. Unlike a general-purpose computer, an embedded system is designed for a single task and typically has no user-accessible operating system.

Embedded systems are everywhere — there are far more embedded processors in the world than general-purpose computers. Your home likely contains dozens: in the microwave, washing machine, central heating thermostat, car, TV remote, and even in your toothbrush if it is electric. They are purpose-built, inexpensive, and reliable because they only need to do one thing well.

Characteristics of Embedded Systems

- Performs **one specific dedicated function** — it does not run arbitrary user programs. A washing machine controller will only ever control the washing machine.
- Program stored in **ROM or flash memory** — runs automatically on power-on without booting an OS. The program is permanent and begins executing immediately.
- **Limited or no user interface** — may have buttons, switches, or LEDs but not a keyboard, mouse, or monitor. Interaction is minimal and specific to the device's function.
- **Real-time response** — must respond within a guaranteed time frame. In a car's anti-lock braking system, a delay of even a fraction of a second could be dangerous. This is why embedded systems often use a Real-Time Operating System (RTOS) or no OS at all.

- **Low power, compact, low cost** — designed to be small, energy-efficient, and inexpensive to mass-produce. Many run on battery power for years (e.g., a pacemaker).
- **Highly reliable** — must work continuously without crashing. A pacemaker cannot "blue screen" or need rebooting. Embedded systems are often tested far more rigorously than desktop software.
- **Difficult to update** — firmware updates may be infrequent or impossible once deployed. Modern embedded systems with internet connectivity (IoT devices) can receive updates, but many older devices cannot.

Examples with Dedicated Functions

Device	Dedicated Function	Why Embedded (Not General-Purpose)?
Washing machine	Controls water temperature, spin speed, and cycle timing based on selected programme	Only needs to run wash cycles. A full PC would be wasteful, expensive, and unreliable for this single task.
Car engine management (ECU)	Monitors sensors (temperature, air flow, RPM) and adjusts fuel injection and ignition timing in real-time	Needs microsecond response times. Must be extremely reliable — engine failure at motorway speed is dangerous.
Anti-lock braking system (ABS)	Detects wheel lock-up and rapidly pulses brakes to maintain traction	Must respond in milliseconds. A general-purpose OS could introduce unpredictable delays (e.g., if it runs a virus scan mid-braking).
Traffic light controller	Cycles through red/amber/green based on timers and vehicle sensor inputs	Runs 24/7 for years without maintenance. Needs to be weatherproof, low-power, and absolutely reliable.
Pacemaker	Monitors heart rhythm and delivers electrical pulses to maintain regular heartbeat	Must run on battery for 5-10 years. A crash or reboot could be fatal. Requires extreme reliability.
Smart thermostat	Monitors temperature sensor and controls heating system to maintain set temperature	Simple, single function. Runs continuously, responds to sensor data, uses minimal power.

Microwave oven	Controls magnetron power level and timer based on user input	Only needs to manage power and timing. A simple microcontroller is far cheaper than a full computer.
GPS navigation unit	Receives satellite signals and calculates position/route	Dedicated to navigation. Real-time processing of satellite signals requires consistent, predictable performance.

Embedded vs General-Purpose Computers

Feature	Embedded System	General-Purpose Computer
Purpose	One dedicated task	Many different tasks (runs any software)
Software	Fixed firmware in ROM/flash	Changeable — user installs/removes programs
User interface	Minimal or none (buttons, LEDs, small display)	Full GUI with keyboard, mouse, monitor
OS	Real-time OS (RTOS) or no OS	Full OS (Windows, macOS, Linux)
Cost	Low (mass-produced, minimal hardware)	Higher (powerful hardware, peripherals)
Power consumption	Very low (milliwatts to watts)	Higher (tens to hundreds of watts)
Processing power	Low to moderate (only needs enough for its task)	High (must handle diverse, demanding applications)
Size	Very small (fits inside other devices)	Larger (standalone devices with peripherals)
Reliability	Extremely high (must run 24/7 for years)	Good, but crashes are tolerable (can reboot)

Examples	Washing machine, car ECU, pacemaker, smart thermostat	Laptop, desktop, server, tablet
----------	---	---------------------------------

Exam Tip: For embedded systems questions, always give a **specific example** and state its **dedicated function**. Do not just say "a washing machine" — say "a washing machine controller that manages water temperature, spin speed, and cycle timing." Also know that embedded systems use ROM (non-volatile, runs automatically) rather than loading software from a hard drive. A common exam question gives a scenario and asks you to explain why an embedded system is more suitable than a general-purpose computer — focus on: single function, low power, low cost, high reliability, real-time response, and compact size.

1.1.6 Secondary Storage Types

Secondary storage is **non-volatile** (data is retained when power is removed) and is used for long-term storage of the OS, programs, and user files. The CPU cannot access secondary storage directly — data must first be loaded into RAM. There are four main types, each with different technologies, speeds, costs, and suitable uses.

Type	Technology	Speed	Capacity	Durability	Cost/GB	Best Use
HDD (Hard Disk Drive)	Spinning magnetic platters with a moving read/write head. Data is stored as magnetic patterns on the platter surface. The head floats nanometres above the	Slow — moving parts mean mechanical seek time (5–10 ms). Sequential read ~100–200 MB/s.	Very high (1–20 TB); cheapest per GB of any technology.	Fragile — the read/write head can crash into the platter if the drive is dropped. Sensitive to vibration and magnetic	Very low (~£0.02–0.04/GB)	Bulk storage, NAS (Network Attached Storage), desktop computers, server data archives, media libraries where cost per GB is the priority.

	spinning platter and changes magnetisation to read/write data. Motor spins the platter at 5400–7200 RPM.			fields. Data can also degrade over many years.		
SSD (Solid State Drive)	NAND flash memory chips — no moving parts. Data is stored as electrical charges trapped in billions of floating-gate transistors. Uses a controller to manage wear levelling (distributing writes evenly across cells to extend lifespan). NAND cells can only be written a finite number of times (TBW — terabytes	Fast — no mechanical movement means near-instant access (~0.05–0.1 ms). SATA SSDs: 500–550 MB/s; NVMe SSDs: 3,000–7,000+ MB/s.	Medium-high (128 GB–4 TB). More expensive per GB than HDDs.	High — no moving parts means shock-resistant, silent, and suitable for mobile devices. However, NAND cells wear out after a finite number of write cycles.	Medium (~£0.08–0.15/GB)	OS drive (fast boot times), laptops (shock resistance), gaming (fast load times), any workload requiring high read/write speeds.

	written rating).					
Optical Disc	A laser reads pits (physical indentations) and lands (flat areas) on a reflective disc surface. CD uses an infrared laser; DVD uses a shorter-wavelength red laser (smaller pits, more data); Blu-ray uses an even shorter blue-violet laser (even more data). The disc spins while the laser tracks across it. Recordable discs (CD-R, DVD-R) use a heat-sensitive dye layer that the laser burns permanently; rewritable discs (RW)	Slow — disc must spin up to speed and the laser must seek the correct track. Sequential read ~150 KB/s (1x CD) to ~72 MB/s (Blu-ray).	Low — CD: 700 MB; DVD: 4.7 GB (single-layer); Blu-ray: 25 GB (single-layer), 50 GB (dual-layer).	Moderate — no moving internal parts, but discs are scratch-sensitive on their data surface. Resistant to magnetic fields. If stored away from light and heat in a case, optical discs can last 50–100+ years (better archival longevity than HDD/SSD). Easily damaged if bent or scratched.	Very low per disc (~£0.20–2.00 per disc regardless of data stored)	Distributing software and media, music/film, long-term archival (especially gold-archival Blu-ray), read-only content distribution, situations where the medium needs to be physically handed to someone.

	use a phase-change alloy that can be re-melted and reset.					
Magnetic Tape	Data is stored as magnetic patterns on a thin plastic tape coated with ferromagnetic material. A read/write head moves across the tape as it winds from reel to reel (or in a cassette cartridge like LTO). Modern LTO (Linear Tape-Open) cartridges store many terabytes on a single tape.	Very slow for random access — to reach data in the middle of the tape, the drive must physically wind past all the preceding tape (sequential access only). However, sequential read/write speed can be fast (250–400 MB/s for LTO-8). Accessing a random file can take minutes.	Very high — LTO-9 cartridges hold up to 18 TB uncompressed (45 TB compressed). Extremely high capacity per unit cost.	High for archival — magnetic patterns on tape are stable for decades if stored correctly (away from heat, moisture, and magnetic fields). No moving internal parts in the cartridge itself. Tape is fragile if physically damaged but has excellent shelf life when cared for.	Extremely low (~£0.002–0.005/GB — the cheapest storage per gigabyte available)	Large-scale long-term backups, disaster recovery archives, cloud provider cold storage (Google, Amazon, Microsoft all use tape for deep archive), broadcasting archives, any situation where data is written once and rarely (if ever) read back.

Comparison Summary — Choosing the Right Storage

Factor	HDD	SSD	Optical	Magnetic Tape
Access type	Random (fast seek)	Random (near-instant)	Sequential / random (slow)	Sequential only (slow random)
Moving parts?	Yes — platter + head	No	Yes — disc spins, laser moves	Yes — tape winds on reels
Typical capacity	1–20 TB	128 GB–4 TB	700 MB–50 GB	Up to 18 TB+ per cartridge
Speed	Medium	Fast to very fast	Slow	Fast sequential, very slow random
Cost per GB	Low	Medium	Very low per disc	Extremely low
Portability	Moderate (external HDDs)	High (USB SSDs)	High (discs easy to post/share)	High (cartridges, but needs special drive)
Ideal for	Bulk desktop storage, cheap large-scale data	OS, apps, laptops, gaming	Distribution, archival, physical copies	Offsite backup, cloud archive, broadcast archive

Scenario: A streaming company needs to archive 10 years of video content rarely accessed but kept for legal reasons.

→ Magnetic tape. Cheapest per GB by far, very high capacity per cartridge, good long-term stability. Sequential access is fine because archives are almost never retrieved.

Scenario: A student needs fast loading times for their laptop's OS.

→ SSD. No moving parts (shock-resistant for portable use), fast boot times, silent. Higher cost per GB is acceptable for smaller capacity.

Scenario: A band wants to distribute 500 copies of their new album.

→ Optical disc (CD). Cheap per unit, easy to distribute physically, recipients need no special hardware beyond a standard CD drive.

Scenario: A video production company needs a fast, large desktop drive.

→ HDD. Very high capacity at low cost. Sequential access for large video files suits HDD characteristics.

Exam Tip: In exam questions on storage, always state whether the storage type has **moving parts** (HDD and optical yes; SSD no; tape yes). Moving parts mean slower access and greater fragility. Magnetic tape is **sequential access** only — it must wind to find data — making it unsuitable for tasks requiring random access but ideal for backup. The four types examiners test most are HDD, SSD, optical, and magnetic tape — know at least two advantages and two disadvantages of each, and be ready to recommend one for a given scenario with justification.